

➤ CommentBox 사용 가이드라인 (팀 공통 컴포넌트)

팀 전원이 모든 게시판에서 동일한 UI/UX의 댓글 기능을 사용할 수 있도록 만든 공통 컴포넌트입니다. 아래 가이드라인을 따라 적용하면 그대로 동작합니다.

▼ 예시 사진

test1234

dd

2025-11-28 12:12 수정 | 삭제

test1234

ddd

2025-11-28 12:12 수정 | 삭제

test1234

dd

2025-11-28 12:12 수정 | 삭제

test1234

o o

2025-11-24 12:09 수정 | 삭제

test1234

o o

2025-11-24 11:17 수정 | 삭제

< 1 2 >

등록

▶ CommentBox란?

- 댓글 목록 조회
- 댓글 등록
- 댓글 수정
- 댓글 삭제
- 댓글 페이지네이션

을 모두 포함하는 댓글 컴포넌트입니다.

이 컴포넌트 하나만 불러오면 댓글 기능이 완성!

▶ 설치/구조

📁 src/Common/Comments/

```
Comments/
├─ CommentBox.jsx
├─ CommentList.jsx
├─ CommentItem.jsx
├─ CommentInput.jsx
├─ CommentPagination.jsx
├─ Comment.styled.js
└─ useComment.js
```

다른 게시판에서는 아래 한 줄만 배치하면 됩니다.

```
<CommentBox ... />
```

▶ 필요한 props

CommentBox는 다음 속성(props)들만 넘겨주면 어떤 게시판에서도 사용 가능

props	설명
boardId	댓글을 조회할 게시글 번호
auth	사용자 인증정보 (isAuthenticated, nickName, accessToken)
fetchAPI	댓글 조회 API 함수
insertAPI	댓글 등록 API 함수
updateAPI	댓글 수정 API 함수
deleteAPI	댓글 삭제 API 함수
mapping	서버 응답 DTO를 CommentBox 구조에 맞게 매핑하는 객체

▶ 적용 예시

▼ 예시 : 인증 게시판 상세보기

```
import { fetchRepliesAPI,
        insertReplyAPI,
        updateReplyAPI,
        deleteReplyAPI
      } from "../../api/activity/activityAPI.js";
import CommentBox from "../../Common/Comments/CommentBox.jsx";

const activityCommentMap = {
  id: "replyNo",
  writer: "writer",
  content: "replyContent",
  date: "enrollDate"
};

<CommentBox
  boardId={id}
  auth={auth}
  fetchAPI={fetchRepliesAPI}
  insertAPI={insertReplyAPI}
  updateAPI={updateReplyAPI}
  deleteAPI={deleteReplyAPI}
  mapping={activityCommentMap}
/>
```

▶ 데이터 매핑 규칙 (mapping)

■ CommentBox는 내부적으로 표준 댓글 구조를 사용

```
{
  id: number,
  writer: string,
  content: string,
  date: string
}
```

서버 응답이 동일 구조가 아닐 경우 mapping으로 매핑

▼ 예시

```
const activityCommentMap = {
  id: "replyNo",
  writer: "writer",
  content: "replyContent",
  date: "enrollDate"
};
```

다른 게시판(예: 공지사항)이 다음 구조면?

```
{
  "commentId": 12,
  "userName": "홍길동",
  "comment": "내용입니다",
  "createdAt": "2025-12-01"
}
```

매핑은 아래와 같이 하면되요

```
const noticeCommentMap = {
  id: "commentId",
  writer: "userName",
  content: "comment",
  date: "createdAt"
};
```

▶ API 요구사항

각 API는 아래 형태여야 함

■ 댓글 조회 API (GET)

```
GET /{boardId}/replies?pageNo=1

response = {
  replies: [],
  pageInfo: { startPage, endPage, maxPage }
}
```

■ 댓글 등록 API (POST)

```
POST /{boardId}/replies
body: { replyContent }
```

■ 댓글 수정 API (PUT)

```
PUT /replies/{replyId}
body: { replyContent }
```

■ 댓글 삭제 API (DELETE)

```
DELETE /replies/{replyId}
```

▶ 사용 요약

```
<CommentBox
  boardId={id}
  auth={auth}
  fetchAPI={fetch}
  insertAPI={insert}
  updateAPI={update}
  deleteAPI={remove}
  mapping={commentMap}
/>
```

▶ UI / UX 맨 윗 예시 사진과 동일하게 유지

- 버튼, 입력창, 페이지네이션 모양 모두 전역 통일
- 너비 900px 기준, 모든 게시판 레이아웃과 통일감 있게함

팀 내 어떤 페이지에도 같은 느낌으로 표시됨

▶ 확인 사항

확인해야할 내용

- ✓ fetchAPI가 boardId, pageNo 두 개 받는지 확인
- ✓ API 응답에 replies 배열이 맞게 오는지
- ✓ mapping 키가 정확한지
- ✓ auth.accessToken이 존재하는지
- ✓ 댓글 기능은 CommentBox 내부 toast가 자동 처리합니다 (onToast 전달 불필요)